ROTEIRO DE ESTUDOS · SERVIDORES LINUX · MÓDULO 09

Segurança: SELinux, AppArmor e hardening

Controle de acesso mandatório com SELinux e AppArmor, políticas criptográficas, auditd, integridade com AIDE, conformidade com OpenSCAP e Ubuntu Security Guide, criptografia de disco com LUKS2 e detecção em logs

Nível	Avançado
Carga sugerida	2 semanas (16–20 h)
Pré-requisitos	Módulos 03, 04, 07 e 08
Relação com RHCSA / LFCS	RHCSA: "Manage security" (modos, contextos, portas e booleanos do SELinux; permissões padrão; SSH por chave; firewalld); LFCS: Users and Groups e Operations

Objetivos do módulo

- Explicar a diferença entre controle de acesso discricionário (permissões) e mandatório (SELinux, AppArmor).
- Operar o SELinux: modos, contextos, restauração de rótulos, portas e booleanos; diagnosticar negações.
- Operar o AppArmor no Ubuntu: perfis em modo enforce e complain.
- Ajustar políticas criptográficas do sistema no RHEL.
- Auditar eventos com auditd e verificar integridade de arquivos com AIDE.
- Avaliar conformidade com OpenSCAP (RHEL) e Ubuntu Security Guide.
- Criptografar volumes com LUKS2.
- Montar uma lista de verificação de hardening e de detecção em logs.

1. Por que controle de acesso mandatório

As permissões tradicionais (Módulo 03) são **discricionárias**: o dono decide, e um processo comprometido age com todos os direitos do usuário que o executa. Um servidor web invadido que roda como `apache` pode ler tudo o que o usuário `apache` lê.

O **controle de acesso mandatório (MAC)** acrescenta uma política definida pelo administrador que limita **o que cada processo pode fazer**, mesmo que as permissões permitam. Se o nginx for comprometido, o SELinux impede que ele leia `/etc/shadow` ou abra conexões que a política não prevê.

	SELinux	AppArmor
Padrão em	RHEL, Fedora, Alma, Rocky	Ubuntu, Debian, SUSE
Modelo	Rótulos (contextos) em todos os arquivos, processos e portas	Perfis por caminho de executável
Abrangência	Todo o sistema	Programas que têm perfil

2. SELinux

2.1 Modos

Modo	Comportamento
Enforcing	Aplica a política e bloqueia o que não é permitido (padrão e recomendado)
Permissive	Não bloqueia, só registra o que seria negado (para diagnóstico)
Disabled	Desligado; não use. Voltar a ligar exige reetiquetar todo o sistema

```
$ getenforce
$ sestatus
# setenforce 0          # permissive até o próximo boot (diagnóstico)
# setenforce 1          # enforcing
# vim /etc/selinux/config   # SELINUX=enforcing  (persistente)
```

Também é possível mudar o modo no boot com os parâmetros de kernel `enforcing=0` ou `enforcing=1`, ou colocar **apenas um domínio** em permissive: `semanage permissive -a httpd_t`.

2.2 Contextos

Todo arquivo, processo e porta tem um **contexto** no formato `usuário:papel:tipo:nível`. Na política padrão (targeted), o que importa no dia a dia é o **tipo**: um processo do tipo `httpd_t` só acessa arquivos de tipos permitidos, como `httpd_sys_content_t`.

```
$ ls -Z /var/www/html
$ ps -eZ | grep nginx
$ id -Z
$ ss -tlpnZ | head
```

2.3 Restaurar e definir contextos de arquivos

Arquivos recebem o contexto do diretório onde são **criados**; um `mv` **mantém** o contexto antigo. Por isso um site movido do `/home` para `/var/www` costuma dar "403 Forbidden".

```
# restorecon -Rv /var/www/html                    # aplica o contexto padrão da política
# semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"   # nova regra persistente
# restorecon -Rv /srv/site                        # aplica a regra
$ semanage fcontext -l | grep /srv/site
# chcon -t httpd_sys_content_t arquivo            # muda só agora; se perde num relabel (evite)
```

> **ATENÇÃO**
>
> Use `semanage fcontext` + `restorecon`, não `chcon`. Alterações com `chcon` desaparecem na próxima reetiquetagem, e você terá um problema "que voltou sozinho" meses depois.

2.4 Portas

```
$ semanage port -l | grep http_port_t
# semanage port -a -t http_port_t -p tcp 8081      # permite o servidor web escutar na 8081
# semanage port -a -t ssh_port_t -p tcp 2222       # SSH em porta alternativa
```

2.5 Booleanos

Interruptores que ligam comportamentos opcionais da política.

```
$ getsebool -a | grep httpd
# setsebool -P httpd_can_network_connect on        # -P torna persistente
# setsebool -P httpd_enable_homedirs on
$ semanage boolean -l | grep -i samba
```

2.6 Diagnosticar negações

```
# ausearch -m AVC -ts recent                      # negações recentes no log de auditoria
# ausearch -m AVC -ts today | audit2why            # explica o motivo
# dnf install setroubleshoot-server
# sealert -a /var/log/audit/audit.log              # análise com sugestões de correção
# journalctl -t setroubleshoot
```

Ordem de raciocínio diante de uma negação: **1) contexto do arquivo está errado?** (`restorecon`); **2) porta não rotulada?** (`semanage port`); **3) existe um booleano?** (`getsebool`); **4) só então** considere um módulo de política próprio com `audit2allow`, revisando o que ele gera.

SEGURANÇA

`audit2allow -M meumodulo` cria uma política que libera exatamente o que foi negado, inclusive o que talvez fosse um ataque. Leia o arquivo `.te` gerado antes de instalar. E nunca "resolva" um problema desligando o SELinux.

3. AppArmor (Ubuntu)

```
# aa-status                                        # perfis carregados e em qual modo
# apt install apparmor-utils
# aa-complain /etc/apparmor.d/usr.sbin.nginx       # só registra (diagnóstico)
# aa-enforce /etc/apparmor.d/usr.sbin.nginx        # aplica
# apparmor_parser -r /etc/apparmor.d/<perfil>      # recarrega após editar
# journalctl -k | grep -i apparmor                 # negações (DENIED)
# aa-logprof                                       # sugere ajustes a partir dos logs
```

NOTA

O Ubuntu 26.04 trouxe **muitos perfis novos** de AppArmor para aplicações. As notas oficiais alertam para possíveis quebras em usos incomuns; quando uma aplicação falhar sem motivo aparente, verifique as negações do AppArmor.

4. Políticas criptográficas (RHEL)

O RHEL define em um só lugar quais protocolos e algoritmos todas as bibliotecas (OpenSSL, GnuTLS, NSS, OpenSSH, Kerberos) aceitam.

```
$ update-crypto-policies --show
# update-crypto-policies --set DEFAULT          # padrão
# update-crypto-policies --set FUTURE           # mais restritiva
# update-crypto-policies --set DEFAULT:SHA1     # política com submódulo (compatibilidade pontual)
```

Política	Uso
`DEFAULT`	Padrão seguro. No RHEL 10, **rejeita cifras TLS com troca de chaves RSA**
`FUTURE`	Mais conservadora, para ambientes que querem antecipar exigências

Política	Uso
`LEGACY`	Compatibilidade com sistemas antigos; evite
`FIPS`	Para conformidade FIPS 140

Algoritmos **pós-quânticos** estão em Technology Preview no RHEL 10; no Ubuntu 26.04, o OpenSSL já inclui ML-KEM, ML-DSA e SLH-DSA, e o OpenSSH usa troca de chaves híbrida pós-quântica por padrão.

> **NOTA**
>
> **FIPS no RHEL 10**: os antigos `fips-mode-setup` e `/etc/system-fips` foram removidos. O modo FIPS é habilitado **na instalação**, com o parâmetro de kernel `fips=1`.

5. Auditoria com auditd

O **auditd** registra eventos de segurança em nível de kernel: execução de comandos, acesso a arquivos sensíveis, mudanças de identidade. É a principal fonte de evidências em investigações.

```
# dnf install audit   |   apt install auditd
# systemctl enable --now auditd

# Regras persistentes em /etc/audit/rules.d/50-lab.rules
-w /etc/passwd -p wa -k identidade
-w /etc/shadow -p wa -k identidade
-w /etc/sudoers -p wa -k privilegio
-w /etc/sudoers.d/ -p wa -k privilegio
-w /etc/ssh/sshd_config -p wa -k ssh
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k comandos_root

# augenrules --load
$ auditctl -l
$ ausearch -k identidade -i
$ aureport --summary
$ aureport -au              # autenticações
$ aureport -x --summary     # executáveis
```

O campo **auid** (audit UID) guarda o usuário que fez o login originalmente, mesmo depois de `sudo` ou `su`. É ele que responde "quem realmente fez isso".

6. Integridade de arquivos com AIDE

```
# dnf install aide   |   apt install aide
# aide --init                                       # [RHEL] cria a base de referência
# mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
# aide --check                                      # compara o estado atual com a base
# aide --update                                     # após mudanças legítimas
```

No Ubuntu, os comandos equivalentes são `aideinit` e `aide --config /etc/aide/aide.conf --check`. Guarde a base de referência fora do servidor (ou em mídia somente leitura): um invasor com root pode alterá-la.

7. Conformidade e linhas de base

- **RHEL**: **OpenSCAP** (`oscap`) com o conteúdo `scap-security-guide`, que traz perfis como CIS, PCI-DSS e STIG. No RHEL 10 a SCAP Workbench foi removida; use `oscap`, `autotailor` ou o Red Hat Lightspeed.
- **Ubuntu**: **Ubuntu Security Guide (USG)**, incluído no Ubuntu Pro, para auditar e aplicar perfis CIS e DISA-STIG.

```
# dnf install openscap-scanner scap-security-guide                    # [RHEL]
$ oscap info /usr/share/xml/scap/ssg/content/ssg-rhel10-ds.xml
# oscap xccdf eval --profile cis_server_l1 \
    --report /root/relatorio-cis.html /usr/share/xml/scap/ssg/content/ssg-rhel10-ds.xml

# pro enable usg && apt install usg                                   # [Ubuntu]
# usg audit cis_level1_server
```

> **DICA**
>
> Comece com **auditoria** e leia o relatório; aplicar um perfil inteiro de uma vez pode quebrar serviços (assim como a linha de base do Windows). Corrija em blocos e teste.

8. Criptografia de disco com LUKS2

LUKS2 é o padrão para novos volumes criptografados no RHEL 10 e no Ubuntu.

```
# cryptsetup luksFormat --type luks2 /dev/vdc                  # APAGA o conteúdo do dispositivo
# cryptsetup open /dev/vdc dados_cripto
# mkfs.xfs /dev/mapper/dados_cripto && mount /dev/mapper/dados_cripto /cripto
# cryptsetup luksDump /dev/vdc
# cryptsetup luksAddKey /dev/vdc                                  # segunda senha (recuperação)
# cryptsetup luksHeaderBackup /dev/vdc --header-backup-file /root/vdc-header.img

# Abertura no boot:
#   /etc/crypttab:  dados_cripto  UUID=<uuid-do-vdc>  none  luks
#   /etc/fstab:     /dev/mapper/dados_cripto  /cripto  xfs  defaults  0 0
```

Para desbloqueio automático em servidores, use **TPM 2.0** (`systemd-cryptenroll --tpm2-device=auto`) ou desbloqueio pela rede com **NBDE (Clevis e Tang)** no RHEL. O Ubuntu 26.04 oferece criptografia de disco com chave no TPM no instalador de desktop.

9. Lista de verificação de hardening

Área	Verificação
Atualizações	Atualizações de segurança automáticas; livepatch onde disponível (Módulo 05)
Contas	Sem login de root por SSH; sudo com regras mínimas; senhas com pwquality e faillock; contas de serviço com `nologin`
SSH	Somente chaves, `AllowGroups`, porta e firewall restritos à rede de administração (Módulo 07)
Serviços	Só o necessário ativo (`systemctl list-unit-files --state=enabled`); portas abertas conferidas (`ss -tulpn`)
Firewall	firewalld ou ufw com negação por padrão
MAC	SELinux em enforcing / AppArmor com perfis em enforce

Área	Verificação
Sistemas de arquivos	`nodev,nosuid,noexec` em `/tmp`, `/var/tmp` e partições de dados; LUKS onde houver dados sensíveis
Logs	Journal persistente, envio para servidor central, auditd com regras para identidade e privilégios
Integridade	AIDE com base guardada fora do servidor
Conformidade	Varredura periódica com OpenSCAP/USG
Unidades systemd	`systemd-analyze security` para endurecer serviços próprios (Módulo 04)

10. Detecção em logs (Blue Team)

O que procurar	Onde e como
Força bruta no SSH	`journalctl -u sshd` (RHEL) ou `-u ssh` (Ubuntu) com "Failed password" e "Invalid user"; `/var/log/secure` ou `/var/log/auth.log`
Uso de sudo	`journalctl _COMM=sudo` ou `_COMM=sudo-rs`; entradas "COMMAND="
Novos usuários e mudanças de grupo	`ausearch -k identidade -i`; journal com `useradd`, `usermod`
Persistência	Novas unidades em `/etc/systemd/system/`, timers, crontabs (`/var/spool/cron`), chaves em `~/.ssh/authorized_keys`
Binários alterados	`rpm -Va` / `dpkg --verify`, AIDE, novos SUID (`find / -perm -4000`)
Negações de MAC	`ausearch -m AVC`, `journalctl -k | grep apparmor`
Conexões estranhas	`ss -tnp`, processos escutando em portas incomuns

LABORATÓRIO

Laboratório 9.1: SELinux na prática (rhel01)

- Mova um site de `/root/site` para `/srv/site`, configure o nginx para servi-lo e observe o erro 403. Diagnostique com `ausearch` e `sealert` e corrija com `semanage fcontext` e `restorecon`.
- Faça o nginx escutar na porta 8081 e corrija com `semanage port`.
- Configure o Samba do Módulo 08 no rhel01 com o contexto e o booleano adequados.
- Coloque o sistema em permissive, gere negações e volte para enforcing. Explique por que nunca se deixa em disabled.

LABORATÓRIO

Laboratório 9.2: auditoria e integridade

- Instale o auditd com as regras da seção 5, crie um usuário, altere o sudoers e identifique tudo com `ausearch`, mostrando o `auid` original.
- Crie a base do AIDE, altere um binário de teste em `/usr/local/bin` e detecte a mudança.
- Rode uma avaliação OpenSCAP (rhel01) e USG (ubu01), escolha 5 itens reprovados e corrija.
- Crie um volume LUKS2 no stor01 que abre no boot com arquivo de chave protegido e guarde o backup do cabeçalho.

Checklist de revisão

- Qual a diferença entre controle de acesso discricionário e mandatório?
- Quais os modos do SELinux e como alterá-los temporária e permanentemente?
- Por que um arquivo movido mantém o contexto antigo? Como corrigir de forma persistente?
- Qual a ordem de investigação de uma negação AVC?
- Como colocar um perfil do AppArmor em complain e voltar para enforce?
- O que muda com a política criptográfica DEFAULT do RHEL 10? Como se habilita FIPS no RHEL 10?
- O que é o `auid` no auditd e por que ele importa?
- Por que a base do AIDE deve ficar fora do servidor?

Referências oficiais

› RHEL 10: usar o SELinux
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_selinux

› RHEL 10: hardening de segurança (políticas criptográficas, auditd, AIDE, OpenSCAP, LUKS)
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/security_hardening

› RHEL 10: proteger redes
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/securing_networks

› Considerações ao adotar o RHEL 10 (FIPS, OpenSCAP, crypto-policies)
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/considerations_in_adopting_rhel_10

› Ubuntu Server: AppArmor
https://ubuntu.com/server/docs/how-to/security/apparmor/

› Visão geral dos recursos de segurança do Ubuntu
https://documentation.ubuntu.com/security/security-features/security-features-overview/

› Ubuntu Pro (USG, CIS, FIPS)
https://ubuntu.com/pro

› Ubuntu 26.04: AppArmor e criptografia pós-quântica
https://documentation.ubuntu.com/release-notes/26.04/summary-for-lts-users/